

Saeyslab Conference 2026: DevOps & Reproducibility

What is DevOps?

Git core concepts

- **Repository:** A folder where Git tracks your project and its history.
- **Commit:** A snapshot of your repository. Contains:
 - The **changes** with respect to the previous commit.
 - A **hash** that identifies it.
 - A reference to its **parent**.
- **Staging area:** a "waiting area" for your changes.
- **Branch:** A pointer to a specific commit.
- **HEAD:** A pointer to the current commit.
- **Working tree:** The state of the files that you see in your directory.
- Git tracks **changes**, not files.



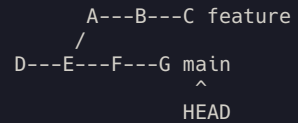
Demo

Basic Git commands

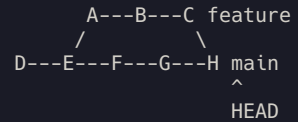
- `git init`: create a repository
- `git add [FILES]`: add files to the staging area
- `git commit [-m MESSAGE]`: create a commit
- `git checkout [HASH | BRANCH]`: move HEAD to a commit/branch or restore files
 - `git switch [HASH | BRANCH]`: move HEAD to a commit/branch
 - `git restore [FILE]`: restore file in the working tree (remove changes)

Merging branches

- `git merge BRANCH`: merges BRANCH into the current branch.
 - It's best to do this when there are no uncommitted changes in the working tree.
 - This creates a *merge commit*: a commit with **2 parents**.
 - If the 2 parents have conflicting changes, these conflicts need to be resolved.
 - If there are no conflicts, the commit has no contents.
- **Challenge:** Use git in terminal



```
$ git merge feature
```



 Demo

GitHub and git remotes

- A **remote** is a copy of the repository on a server somewhere
- A repository can have multiple remotes
- `git remote add [NAME] [URL]`: adds URL as a remote (typically `origin`)
- `git push`: uploads commits
 - A branch gets assigned an **upstream branch** to track (synchronize)
- `git pull`: downloads commits
 - Will also perform `git merge` if necessary
- GitHub is **just one option** for storing your repositories
 - Alternatives include **GitLab**, **Bitbucket**, **Forgejo**, **Codeberg**, ...
- GitHub adds many bells and whistles to your git repo:
 - **Issues**: a kind of "support ticket" for your repo
 - **Pull requests**: should be called "merge requests"
 - A pull request is a *type* of issue
 - **GitHub Actions**: runs code on certain events
 - Wiki, discussions, releases, ...

 Demo

Resources

- <https://learngitbranching.js.org/>
- <https://ohshitgit.com/>

Dependency Management

- Where is `ls`?

```
echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
```

- In Python: `sys.path`
- In R: `.libPaths()`

 Demo

Dependency Management

- Dependency: any code you didn't write
- Dependencies can have their own dependencies
- Projects can have conflicting dependencies
 - This is why virtual environments exist



Dependency Management

- Reproducibility: dependencies must be *pinned*
- **Question:** Why split dependencies into explicit dependencies and a lockfile?
 - Maximizing compatibility vs. reproducibility
- Dependencies can introduce overhead
- Tips:
 - Use a virtual environment (`uv` in Python, `Renv` in R)
 - Use a lockfile (this disqualifies `conda`, `mamba`)
 - Minimize the number of dependencies
 - For reproducibility: pin your dependencies strictly
 - For compatibility: specify ranges of dependency versions

Project Structure

- Core idea:
 - Separate library code from scripts
 - Separate raw data, preprocessed data, results and figures
 - Use a virtual environment
- Tip: try to autogenerate your project structure
 - Template repository
 - `uv init --package`
 - `cookiecutter`

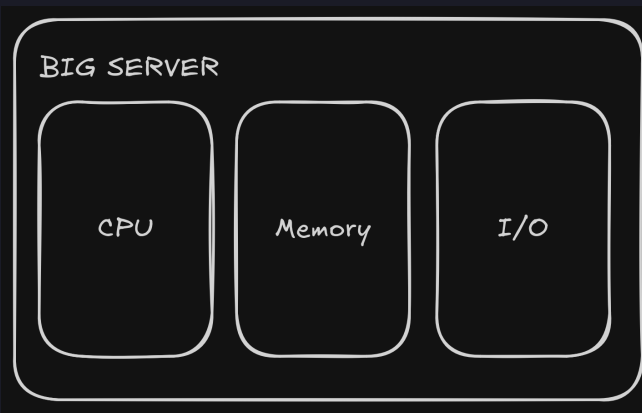
Project Structure: example

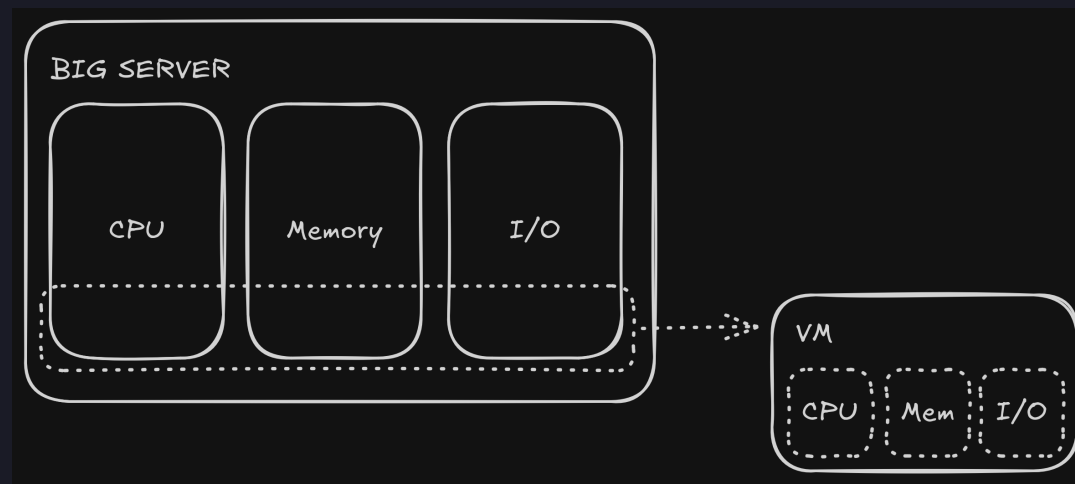
 Python

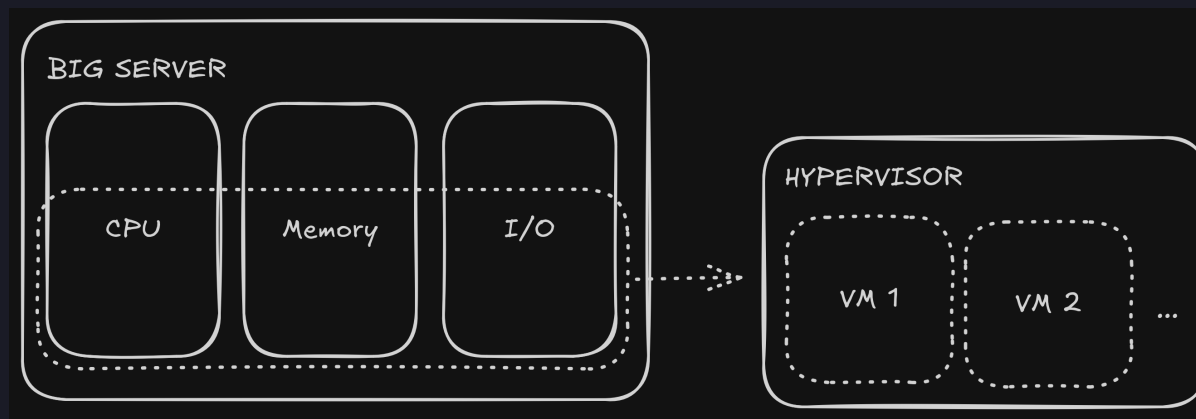
```
pyproject.toml
uv.lock
README.md
.venv/
scripts/
├─ 1_load_data.py
├─ 2_run_analysis.py
├─ 3_plot_results.py
notebooks/
├─ sandbox.ipynb
tests/
├─ test_datasets.py
src/
├─ my_package/
│   ├── util.py
│   ├── datasets.py
│   └── plot.py
data/
├─ raw/
└─ preprocessed/
output/
├─ 2026-08-16_all_datasets/
└─ 2026-08-18_visiumhd/
plot/
├─ 2026-08-18_visiumhd/
```

 R

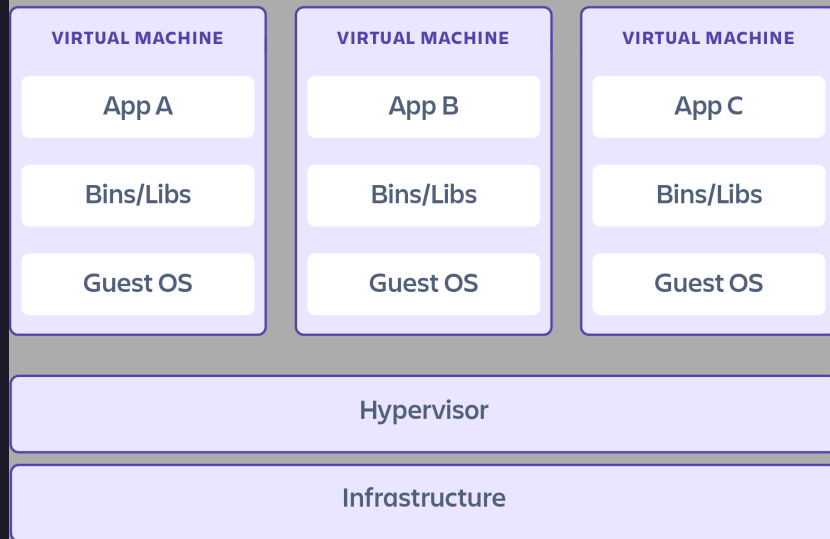
```
project.Rproj
renv.lock
README.md
renv/
scripts/
├─ 1_load_data.R
├─ 2_run_analysis.R
├─ 3_plot_results.R
vignettes/
├─ sandbox.Rmd
tests/
├─ test_datasets.R
R/
├─ util.R
├─ datasets.R
└─ plot.R
data/
├─ raw/
└─ preprocessed/
output/
├─ 2026-08-16_all_datasets/
└─ 2026-08-18_visiumhd/
plot/
├─ 2026-08-18_visiumhd/
```

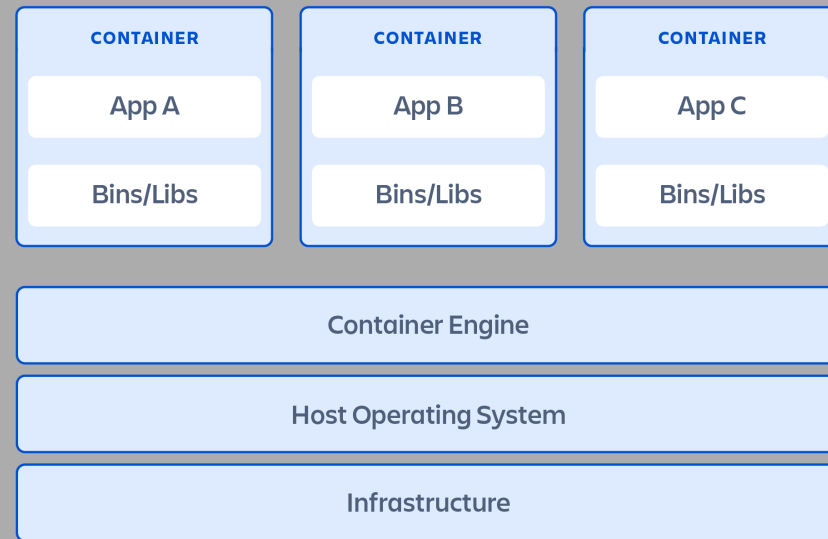




Virtual machines



Containers



The Dockerfile

Folder structure

```
Dockerfile
coffee-recipe/
├── prepare_beans.sh
└── brew_coffee.sh
```

Dockerfile

```
FROM ubuntu:latest
RUN apt-get update && apt-get -y install coffee-maker
COPY ./coffee-recipe /coffee-recipe
WORKDIR /coffee-recipe
RUN ./prepare-beans.sh
CMD ["./brew_coffee.sh"]
```


Advanced concepts

- Persistent storage: *volume mounts*
- Using docker for development: *bind mounts, development containers*
- Orchestrating multiple containers: *docker compose*
- **Challenges:**
 - Docker In A Month of Lunches book
 - Docker Workshop
 - Use a development container
 - Learn about Github Actions

Reproducibility: general tips

- Full automation of results and figures
- Avoid hardcoding
 - Hardcoded paths kill reproducibility
 - Use a `.env` file if you need to
- Use external configuration files
- Time stamp your results and plots in the file name
- Save experiment metadata along with results
- Scripts should have well-defined inputs and outputs
 - `raw_data.csv => preprocess.{py,R} => preprocessed.h5ad`
 - `preprocessed.h5ad => analysis.{py,R} => results.csv`
 - `results.csv => plot.{py,R} => plot.png`
- Pin versions and commit your lockfile
 - For multi-language projects: consider devcontainers or `pixi`

Reproducibility: notebooks

Demo

- Nonlinear execution kills reproducibility
- Notebooks encourage code duplication
- Notebooks don't play nice with version control
 - Clear their outputs before committing
- Notebooks don't play nice with slurm
- Use only for drafting/experimenting or to make vignettes

Reproducibility: containers



Demo

Sources

- Dependency management
 - <https://linuxvox.com/blog/see-path-linux/>
- Project structure
 - <https://www.r-bloggers.com/2018/08/structuring-r-projects/>
 - https://intro2r.com/dir_struct.html
 - https://tmieno2.github.io/WritingJournalArticleRmarkdown/C1_1_ProjectOrganization.html
 - <https://r-statistics.co/R-Project-Structure.html>
- Containers
 - <https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms>
 - <https://www.youtube.com/watch?v=Ud7Npgi6x8E>
 - Docker In A Month of Lunches, Elton Stoneman
 - <https://docs.docker.com/get-started/introduction/>
 - <https://docs.docker.com/get-started/workshop/>
- <https://missing.csail.mit.edu/>